

Capacitaciones Lab IV

Objetivo

El objetivo de la capacitación es que los integrantes del equipo se familiaricen con las tecnologías, herramientas y prácticas de desarrollo que serán utilizadas durante la ejecución del proyecto de Laboratorio IV.

Para ello, se desarrollará una aplicación reducida que permita ejercitar la construcción de un frontend, un backend, una base de datos y la infraestructura necesaria para ejecutar los componentes localmente.

Descripción de la capacitación

Durante la capacitación se desarrollará una aplicación web que permita crear reuniones entre distintos usuarios e incorporar videollamadas mediante Jitsi Meet.

La aplicación estará compuesta por:

- Un backend desarrollado con Java y Spring Boot.
- Un frontend desarrollado con React, TypeScript y Vite.
- Una base de datos PostgreSQL.
- Contenedores Docker para ejecutar los diferentes componentes.
- Repositorios remotos en GitHub.

Este ejercicio permitirá trabajar sobre conceptos similares a los requeridos por Study Arena, particularmente la creación de espacios compartidos, la participación de distintos usuarios y la comunicación en tiempo real. Estas capacidades están relacionadas con las épicas de sesiones de estudio y funcionalidades sociales definidas para el producto.

Alcance funcional

La aplicación desarrollada durante la capacitación deberá permitir:

1. Registrar usuarios.
2. Autenticar usuarios.
3. Crear una reunión.
4. Consultar las reuniones disponibles.
5. Unirse a una reunión.
6. Acceder a una videollamada asociada a la reunión.
7. Ejecutar la aplicación completa localmente mediante Docker Compose.

El objetivo es construir un proyecto acotado que permita adquirir experiencia con las tecnologías y decisiones técnicas que posteriormente serán utilizadas en el producto principal.

Repositorios y control de versiones

Se utilizará GitHub como plataforma para alojar y compartir los repositorios remotos.

El código se deberá organizar de la siguiente manera:

- Un único repositorio que contenga el proyecto de backend y el de frontend.

La alternativa seleccionada deberá mantener una separación clara entre los componentes y permitir que cada uno pueda compilarse y ejecutarse de manera independiente.

Durante la capacitación se deberán practicar las siguientes operaciones:

- Clonar un repositorio.
- Crear ramas de trabajo.
- Registrar cambios mediante commits.
- Publicar ramas en el repositorio remoto.
- Crear pull requests.
- Resolver conflictos simples.
- Integrar cambios en la rama principal.

Backend

El backend será desarrollado utilizando:

- Java.
- Spring Boot.
- Spring Web.
- Spring Data JPA.
- Spring Security.
- JSON Web Tokens.
- PostgreSQL.
- Gradle con sintaxis Groovy.
- Docker.

El proyecto deberá crearse mediante Spring Initializr (<https://start.spring.io/>) y utilizar Gradle Groovy como herramienta de construcción y administración de dependencias.

Estructura sugerida

El backend deberá mantener separadas las diferentes responsabilidades de la aplicación.

Una estructura inicial posible es:

```
src/main/java
├── edu.studyarena.training
│   ├── configuration
│   ├── controller
│   ├── dto
│   ├── entity
│   ├── exception
│   ├── repository
│   ├── security
│   └── service
```

Las responsabilidades principales serán:

- **Controller:** exponer los endpoints HTTP.
- **Service:** implementar los casos de uso y reglas de negocio.
- **Repository:** acceder a la base de datos.
- **Entity:** representar las entidades persistidas.
- **DTO:** definir los contratos de entrada y salida.
- **Security:** implementar la autenticación y validación de tokens.
- **Configuration:** contener las configuraciones generales de Spring.

Modelo inicial

La aplicación podrá comenzar con las siguientes entidades.

Usuario

Representa a una persona registrada en la aplicación.

Atributos mínimos:

- Id
- Nombre.
- Correo electrónico.
- Contraseña almacenada de manera segura (hasheada).

Reunión

Representa una reunión creada por un usuario.

Atributos mínimos:

- Id.
- Nombre.
- Descripción.
- Fecha y hora.
- Identificador de la sala de Jitsi Meet.
- Usuario creador.
- Fecha de creación.

Paso a paso

Crear el proyecto

Crear un proyecto desde Spring Initializr (<https://start.spring.io/>) con las dependencias necesarias para:

- Exponer una API REST.
- Acceder a PostgreSQL.
- Persistir entidades con JPA.
- Validar los datos recibidos.
- Implementar autenticación con Spring Security.

Implementar las entidades

Crear las entidades User y Meeting, junto con sus relaciones.

La entidad de reunión deberá identificar al usuario que la creó y almacenar la información necesaria para acceder a la sala de videollamada.

Implementar los repositorios

Crear interfaces basadas en Spring Data JPA para acceder a las entidades persistidas.

Como mínimo se deberá poder:

- Buscar un usuario por correo electrónico.
- Guardar un nuevo usuario.
- Guardar una reunión.
- Listar reuniones.
- Buscar una reunión por identificador.

Implementar los servicios

Crear servicios independientes para los casos de uso principales.

Como mínimo:

- Registro de usuarios.

- Autenticación.
- Creación de reuniones.
- Consulta de reuniones.
- Consulta de una reunión específica.

Los controladores no deberán acceder directamente a los repositorios. La lógica de negocio deberá permanecer dentro de los servicios correspondientes.

Crear los endpoints en los controllers

Endpoints mínimos sugeridos:

POST /api/auth/register
 POST /api/auth/login
 GET /api/meetings
 GET /api/meetings/{id}
 POST /api/meetings

Las respuestas deberán utilizar códigos HTTP adecuados y contratos consistentes.

Implementar la autenticación

La autenticación se realizará mediante Spring Security y JWT (ejemplo de tutorial: <https://medium.com/somos-pragma/gu%C3%ADa-de-implementaci%C3%B3n-jwt-para-la-autenticaci%C3%B3n-en-java-db47b04eda54>).

El flujo esperado será:

1. El usuario envía su correo electrónico y contraseña.
2. El backend valida las credenciales.
3. El backend genera un token JWT.
4. El frontend incluye el token en las solicitudes protegidas.
5. El backend valida el token antes de permitir el acceso al recurso.

Los endpoints de registro y autenticación serán públicos. Los endpoints relacionados con reuniones requerirán autenticación.

Integrar Jitsi desde el backend

La integración con Jitsi no implica que el backend procese el audio o el video. Esa comunicación ocurre entre el navegador y Jitsi. La responsabilidad del backend es administrar la reunión, definir una sala estable y, cuando se utilice una instalación protegida, autorizar el ingreso mediante un token temporal.

La capacitación utilizará Jitsi as a Service (JaaS). Las reuniones se ejecutarán en la infraestructura de Jitsi mediante el dominio 8x8.vc.

Antes de implementar el código, cada grupo deberá disponer de una aplicación creada en la consola de JaaS. De allí se obtienen tres datos:

- AppID: identifica la aplicación y normalmente comienza con vpaas-magic-cookie-.
- Key ID (`kid`): identifica la clave pública que JaaS utilizará para verificar la firma del token.
- Clave privada RSA: permite al backend firmar los tokens de acceso. JaaS no necesita recibir esta clave.

El flujo de confianza es el siguiente:

1. La clave pública se registra en JaaS.
2. La clave privada se conserva solamente en el backend.
3. El backend firma un JWT diferente para cada participante.
4. JaaS utiliza la clave pública asociada al `kid` para verificar la firma.
5. Si la firma y los datos del token son válidos, permite el ingreso a la sala.

Pasos a considerar:

Modelar la sala

La reunión deberá almacenar un identificador de sala. El identificador debe generarse una sola vez cuando se crea la reunión y debe ser único y difícil de adivinar. Además, tiene que contener caracteres seguros para una URL.

Separar la configuración del código

El dominio de Jitsi y las credenciales del proveedor deben configurarse mediante variables de entorno y el archivo en la carpeta de Config. No deben escribirse directamente en controllers, services ni archivos versionados.

Crear un servicio de acceso a Jitsi

Crear un servicio específico evita mezclar la integración externa con el controller de reuniones. Un ejemplo del contrato podría ser

```
public interface VideoConferenceAccessService {

    VideoConferenceAccess createAccess(Meeting meeting, User user);

}
```

El resultado puede ser un DTO como el siguiente:

```
public record VideoConferenceAccess (
    String domain,
    String roomName,
    String token,
    Instant expiresAt
) {}
```

Definir el endpoint de acceso

Agregar un endpoint protegido que genere el acceso justo antes de ingresar a la videollamada (ej: POST /api/meetings/{meetingId}/access)

La respuesta podría ser del estilo

```
{
  "domain": "8x8.vc",
  "roomName": "<app-id>/programacion-grupo-4-f7a91c2d",
  "token": "<jwt-de-jitsi>",
  "expiresAt": "2026-08-02T20:10:00Z"
}
```

Antes de generar la respuesta, el backend deberá:

1. Obtener el usuario desde el contexto de Spring Security.
2. Buscar la reunión por su identificador.
3. Verificar que el usuario puede ingresar.
4. Comprobar, si corresponde, que la reunión se encuentra dentro del horario permitido.
5. Recuperar el identificador de sala persistido.
6. Generar un token de corta duración.

Este token es diferente del JWT utilizado para autenticarse en la aplicación. El Jwt de jitsi permite ingresar a una sala de videollamada y es validado por jitsi.

En JaaS el token se firma con RSA y contiene información similar a la siguiente:

```
{
  "aud": "jitsi",
  "iss": "chat",
  "sub": "<app-id>",
  "room": "programacion-grupo-4-f7a91c2d",
  "nbf": 1785700000,
  "exp": 1785700600,
  "context": {
    "user": {
      "id": "<id-interno-del-usuario>",
      "name": "Nombre del usuario",
      "email": "usuario@example.com",
      "moderator": false
    },
    "features": {
      "recording": false,
      "livestreaming": false,
      "transcription": false
    },
    "room": {
      "regex": false
    }
  }
}
```

Manejo de errores

Implementar un mecanismo centralizado para transformar excepciones en respuestas HTTP.

Como mínimo deberán contemplarse:

- Datos de entrada inválidos.
- Credenciales incorrectas.
- Usuario no encontrado.
- Reunión no encontrada.
- Usuario ya registrado.
- Token ausente o inválido.
- Errores internos no controlados.

Crear el Dockerfile

Un dockerfile es un archivo utilizado por docker para poder construir la imagen que utilizaremos en los contenedores.

El backend deberá contar con un Dockerfile (ejemplo: <https://spring.io/guides/gs/spring-boot-docker>) que permita:

1. Compilar la aplicación.
2. Generar el archivo ejecutable.
3. Construir una imagen final.
4. Ejecutar el backend dentro de un contenedor.

Crear el Docker-Compose

Se utilizará Docker Compose para levantar localmente el stack completo.

El stack deberá incluir:

- Backend.
- PostgreSQL.

Ejemplo conceptual:

```
services:
  database:
    image: postgres:<version-fijada>

  backend:
    build:
```



```
context: ./backend
dockerfile: Dockerfile
ports:
  - "${EXTERNAL_APP_PORT}:8080"
depends_on:
  - database
```

Frontend

El frontend será desarrollado utilizando:

- React.
- TypeScript.
- Vite.
- Axios para solicitudes HTTP.
- Jitsi Meet External API.

Estructura sugerida

Una estructura inicial posible es:

```
src
├── api
├── components (van a ser basados en shadcn)
├── contexts
├── hooks
├── layouts
├── pages
├── routes
├── services
├── types
└── utils
```

Pantallas mínimas

El frontend deberá incluir:

1. Pantalla de registro.
2. Pantalla de inicio de sesión.
3. Listado de reuniones.
4. Formulario para crear una reunión.
5. Vista de detalle de una reunión.
6. Vista de videollamada.

Paso a Paso

Crear el proyecto

Crear el proyecto utilizando Vite con la plantilla de React y TypeScript (npm create vite).

La aplicación deberá iniciar correctamente en el entorno local antes de agregar nuevas dependencias.

Definir la estructura

Crear las carpetas principales y separar:

- Componentes visuales.
- Pantallas.
- Tipos.
- Servicios.
- Integración con la API.
- Estado de autenticación.
- Configuración de rutas.

Configurar variables de entorno

Configurar la dirección del backend mediante una variable de entorno.

Por ejemplo:

VITE_API_URL

No se deberán incluir direcciones de entornos particulares directamente dentro de los componentes.

Implementar el registro

Crear un formulario que permita ingresar:

- Nombre.
- Correo electrónico.
- Contraseña.
- Confirmación de contraseña.

El formulario deberá validar los datos antes de enviarlos al backend y mostrar los errores correspondientes.

Implementar el inicio de sesión

Crear un formulario para enviar las credenciales al backend.

Cuando la autenticación sea exitosa:

1. Se recibirá el token JWT.
2. Se almacenará la información necesaria para mantener la sesión.
3. El usuario será redirigido al listado de reuniones.
4. Las solicitudes protegidas incluirán el token.

Implementar el listado de reuniones

Consultar el backend y mostrar las reuniones disponibles.

Cada elemento deberá mostrar, como mínimo:

- Nombre.
- Descripción.
- Fecha y hora.
- Usuario creador.
- Acción para acceder al detalle.

Implementar la creación de reuniones

Crear un formulario que permita ingresar los datos de una reunión.

Al confirmar:

1. Se enviará la información al backend.
2. El backend generará o almacenará el identificador de la sala.
3. El frontend redirigirá al detalle de la reunión creada.

Integrar Jitsi Meet

Incorporar una videollamada de Jitsi as a Service mediante `@jitsi/react-sdk`.

La sala deberá identificarse a partir de la información recibida desde el backend. Dos usuarios que ingresen a la misma reunión deberán acceder a la misma sala.

La integración deberá permanecer encapsulada en un componente específico para evitar que las pantallas dependan directamente de los detalles de Jitsi Meet.

La implementación deberá separar la navegación, el acceso a la API y la integración con Jitsi. La capacitación propone utilizar Axios, autenticación Bearer y componentes con responsabilidades acotadas.

El flujo esperado es:

1. El usuario inicia sesión en la aplicación.
2. El frontend consulta y muestra las reuniones disponibles.

3. El usuario selecciona una reunión y accede a su detalle.
4. Al presionar "Unirse", el frontend solicita al backend un acceso temporal para esa reunión.
5. El backend responde con el dominio, el nombre de sala y el JWT de Jitsi.
6. El frontend monta el componente de Jitsi con esos datos.
7. JaaS verifica el token y permite el ingreso.

Paso 1: Instalar el SDK

```
npm install @jitsi/react-sdk
```

Paso 2: definir los tipos del frontend

Crear un tipo para las reuniones que represente el contrato utilizado por la interfaz

Paso 3: realizar los services del frontend que buscan las meetings y piden el acceso de jitsi. Similar a

```
export async function getJitsiAccess(  
  meetingId: string,  
) : Promise<JitsiAccess> {  
  const response = await api.post<JitsiAccess>(  
    `/meetings/${meetingId}/access`,  
  );  
  return response.data;  
}
```

Paso 4: crear un componente o página que coordine el ingreso.

No se deben pedir tokens al cargar el listado: podría vencer antes de ser utilizado y se generarían credenciales innecesarias. Debe solicitarse inmediatamente antes de montar la videollamada.

Paso 5: crear componente de viewing de meeting basado en el sdk de jitsi

Código de referencia de uso del componente

```
<JitsiMeeting  
  domain={access.domain}  
  roomName={access.roomName}  
  jwt={access.token}  
  configOverwrite={{  
    startWithAudioMuted: true,  
    startWithVideoMuted: true,  
  }}  
  getIFrameRef={(iframe) => {  
    iframe.style.width = '100%';  
    iframe.style.height = '100%';  
  }}  
>
```

```
} }  
/>
```

Para JaaS:

- domain será normalmente ``8x8.vc``.
- roomName deberá incluir el namespace del AppID, por ejemplo `<app-id>/<sala>`.
- jwt será el token temporal firmado por el backend.